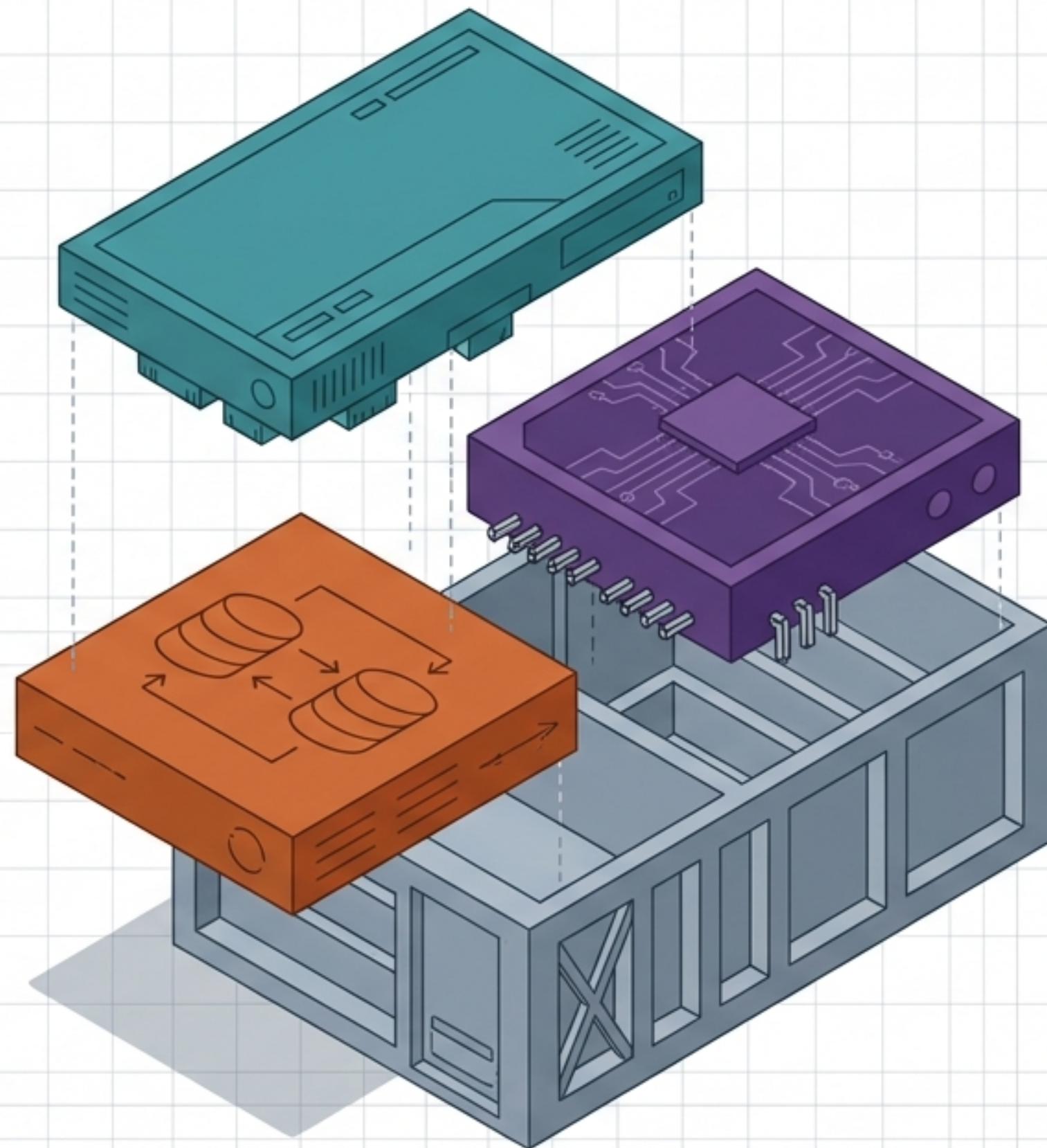


Despliegue de Servidores Web: Arquitectura y Contenedores

Del análisis teórico a la orquestación moderna en la nube.



Una guía visual para el ensamblaje de la pila web (Apache, MariaDB, PHP).

El Ciclo de Vida del Despliegue



Análisis (Fase 1)

Definición de requisitos funcionales y de infraestructura.



Preparación (Fase 2)

Acondicionamiento del sistema operativo host y la red.



Montaje (Fase 3)

Integración de los módulos (Web, Datos, Scripting) vía contenedores.



Validación (Fase 4)

Comprobación del ecosistema y documentación de la arquitectura.

Fase 1: Matriz de Análisis de Requisitos

Requisitos Funcionales	Requisitos No Funcionales	Requisitos de Infraestructura
Definición: Qué debe hacer el sistema. Ejemplos: Procesamiento de formularios, gestión de usuarios, almacenamiento de registros.	Definición: Cómo debe rendir el sistema. Ejemplos: Alta disponibilidad, tiempos de respuesta, facilidad de mantenimiento.	Definición: Entorno y compatibilidad. Ejemplos: Compatibilidad del SO, recursos Cloud, marco legal.

Fase 2: Preparación del Entorno

Servidor Tradicional (Bare-Metal)



- Copias de seguridad físicas en discos externos.
- Desfragmentación del disco duro para agilizar lectura.
- Desmontaje/montaje de dispositivos RAID físicos.

Instancia Cloud Moderna



- Snapshots instantáneos del estado del servidor.
- Provisionamiento dinámico de volúmenes SSD (sin necesidad de desfragmentar).

¡Regla de Oro! Solo un servidor web puede escuchar el puerto 80/443 a la vez.
Evitar conflictos de puertos es el primer paso antes de desplegar.

Arquitectura de Procesamiento: Cliente vs. Servidor



Cliente (Navegador)



- Tecnologías: HTML, CSS, JavaScript.
- Función: Interpretación visual, validación de formularios, efectos de interfaz.
- Ventaja: Descarga de trabajo al servidor; respuesta inmediata al usuario.

Servidor (Backend Cloud)

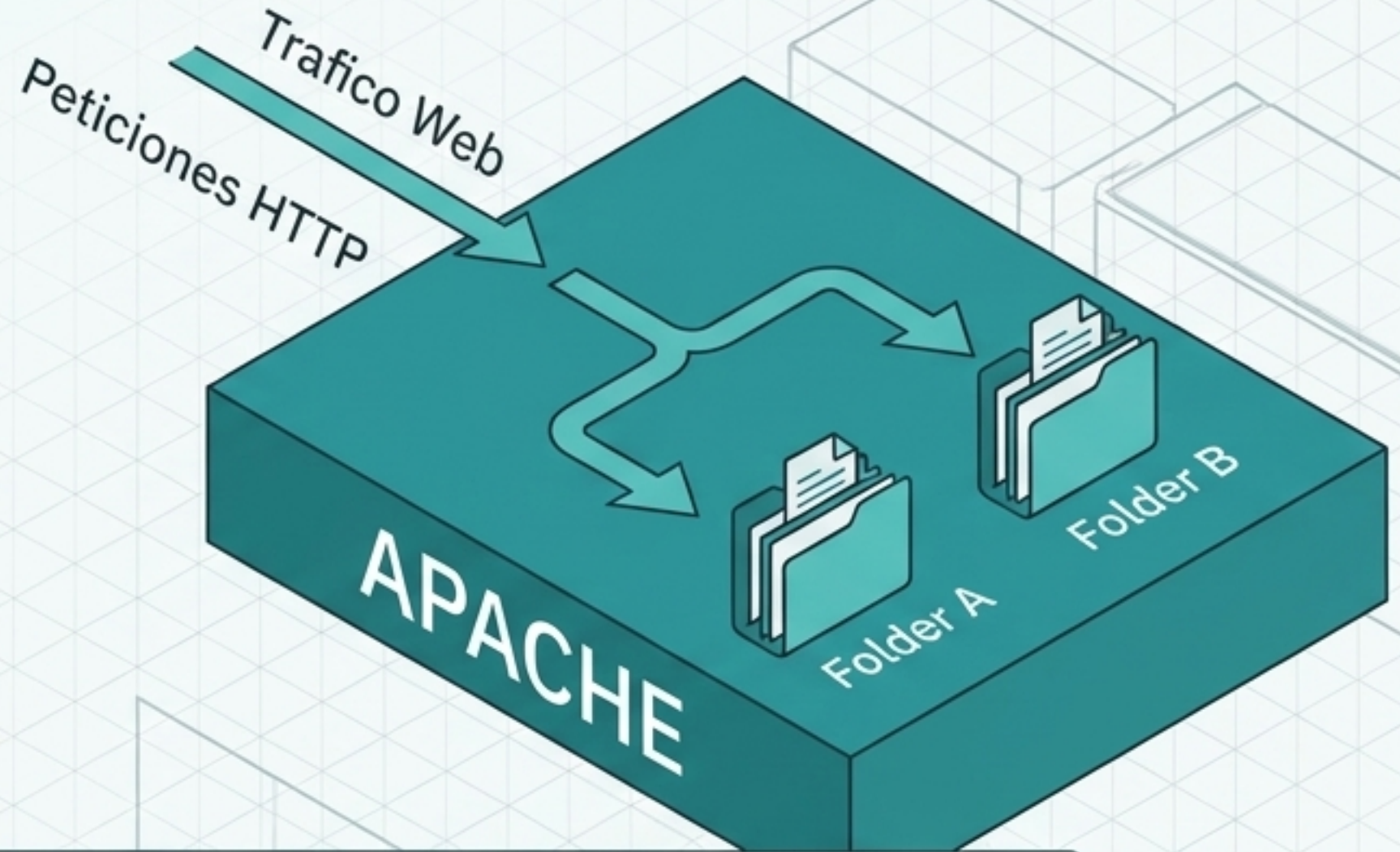


- Tecnologías: Apache, PHP, MariaDB.
- Función: Acceso a bases de datos, lógica de negocio compleja, generación dinámica de HTML.
- Seguridad: El código fuente nunca es visible para el cliente.

Capa Web: El Servidor Apache

El controlador de tráfico HTTP.

- Modular, multiplataforma y de código abierto.
- Recibe peticiones HTTP y sirve documentos al navegador.



¿Cómo aloja una empresa múltiples webs en un solo servidor?

Mediante Virtual Hosts, que permiten múltiples nombres de dominio bajo una misma instancia de Apache.

Capa de Datos: Sistema Gestor (SGBD)

El motor de almacenamiento relacional.

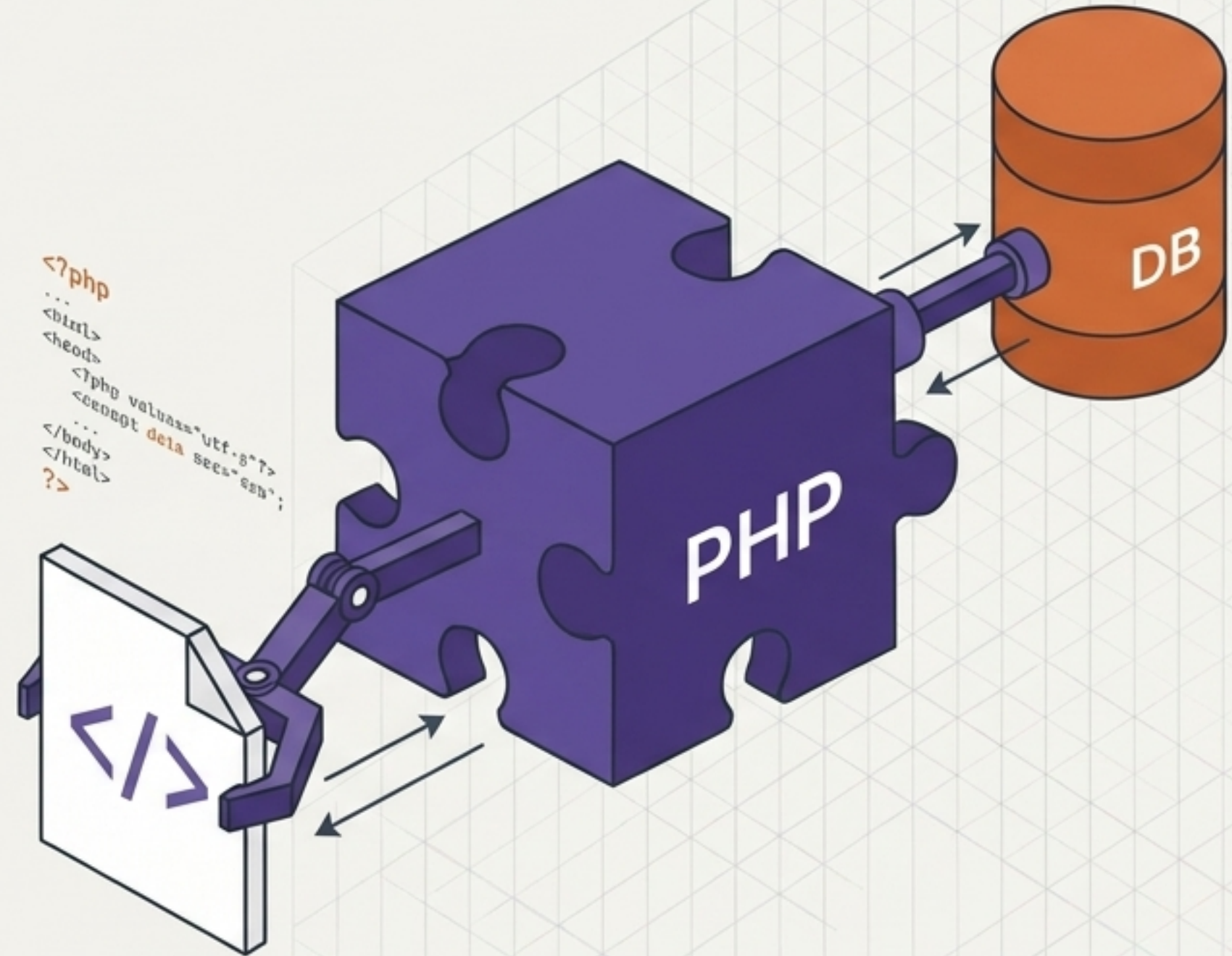
- Creado como reemplazo directo y de código abierto de MySQL.
- Cumple estrictamente con el estándar SQL.
- Alta velocidad de acceso a datos.
- Totalmente integrado con funciones nativas de PHP.



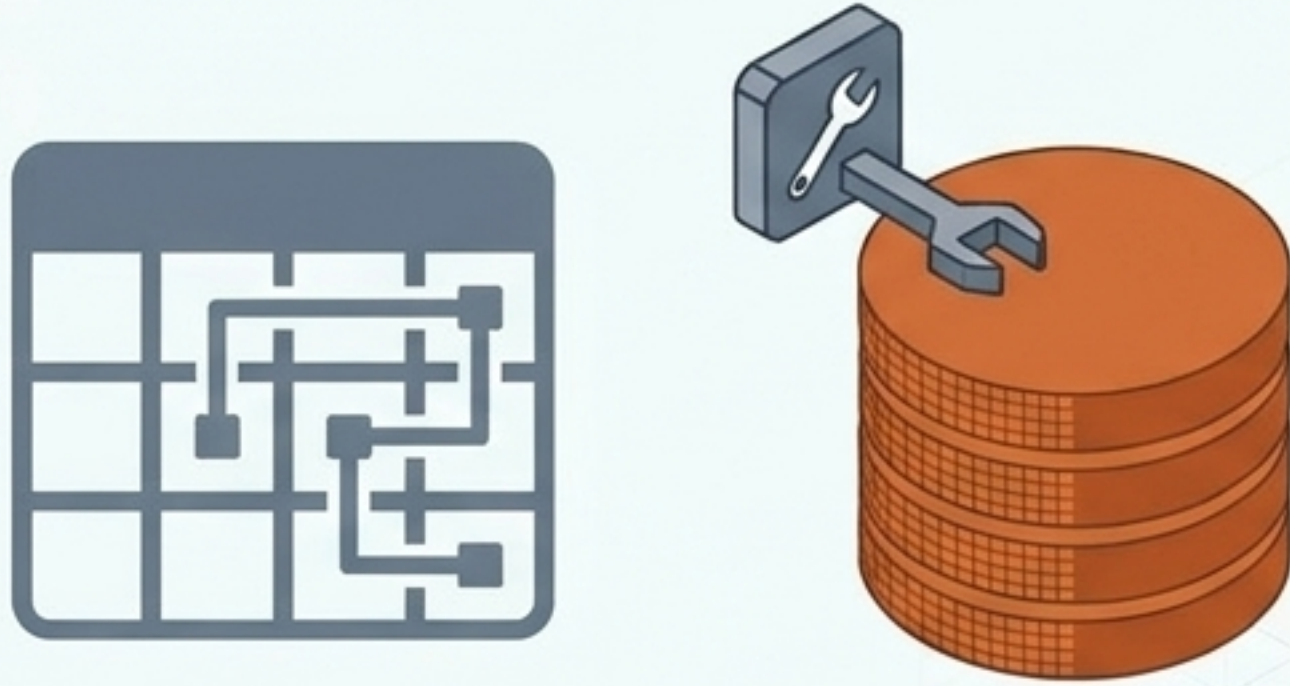
Capa Lógica: Lenguaje de Scripting (PHP)

El puente dinámico del lado del servidor.

- Se incrusta directamente en el código HTML.
- Traduce la lógica compleja y las consultas a la base de datos en HTML puro.
- Transforma páginas web estáticas en aplicaciones interactivas.

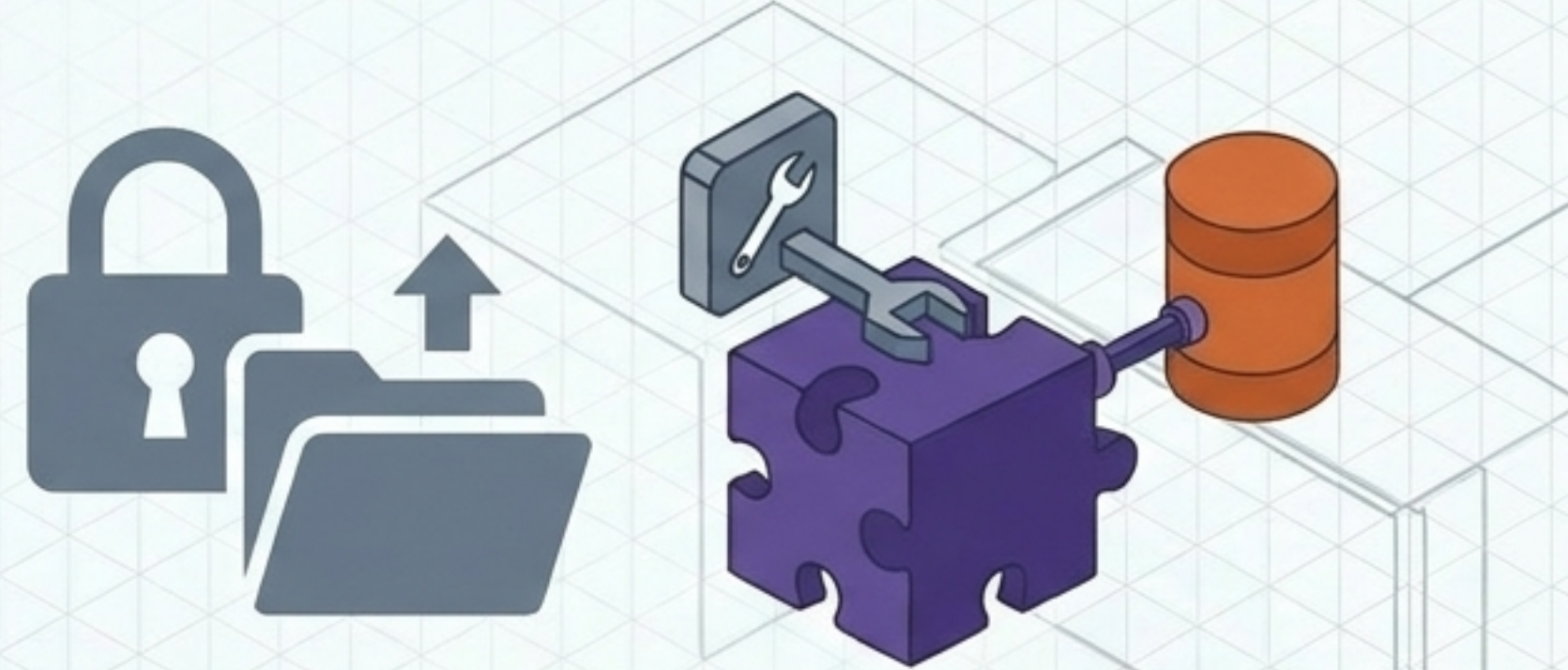


Componentes de Gestión y Soporte



phpMyAdmin

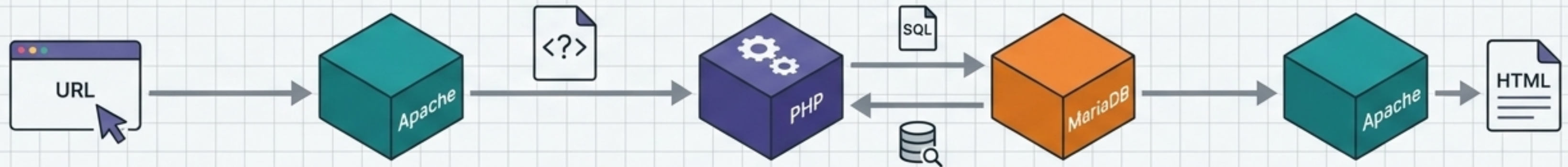
Herramienta gráfica escrita en PHP. Permite la administración visual del servidor MariaDB sin depender exclusivamente de la consola de comandos.



Protocolos de Administración (SSH / FTP)

Módulos para transferencia de ficheros y control remoto seguro del servidor de producción.

Síntesis: El Flujo de la Petición Dinámica



1. Petición

El cliente solicita una URL a través del navegador.

2. Recepción

Apache localiza el archivo. Al ver la extensión .php, llama al módulo PHP.

3. Procesamiento

PHP ejecuta los scripts y solicita datos.

4. Consulta

MariaDB procesa la solicitud SQL y devuelve los datos a PHP.

5. Respuesta

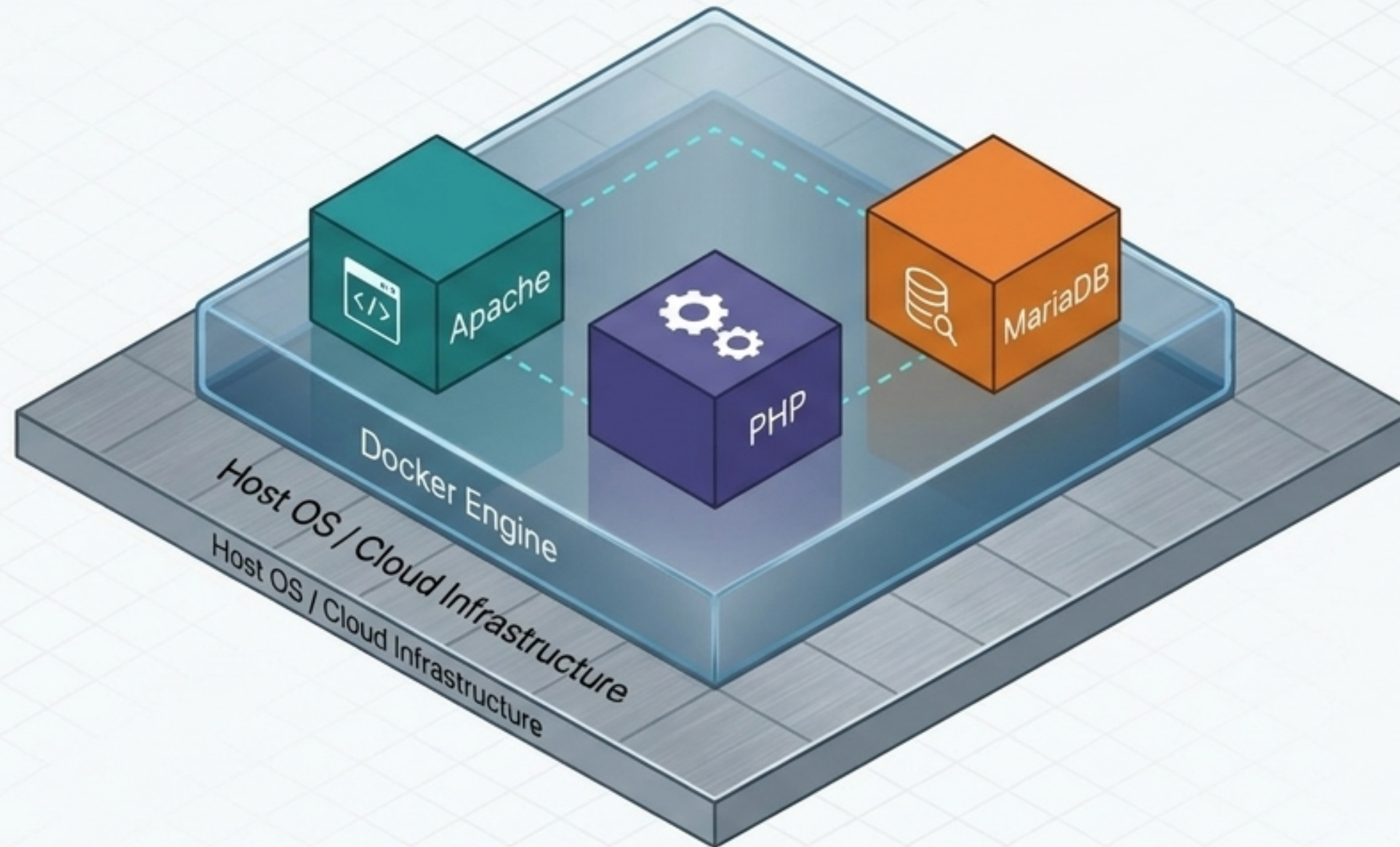
PHP compila todo en un documento HTML final y Apache se lo sirve al cliente.

La Evolución de los Entornos Integrados

De instaladores monolíticos locales a contenedores en la nube.

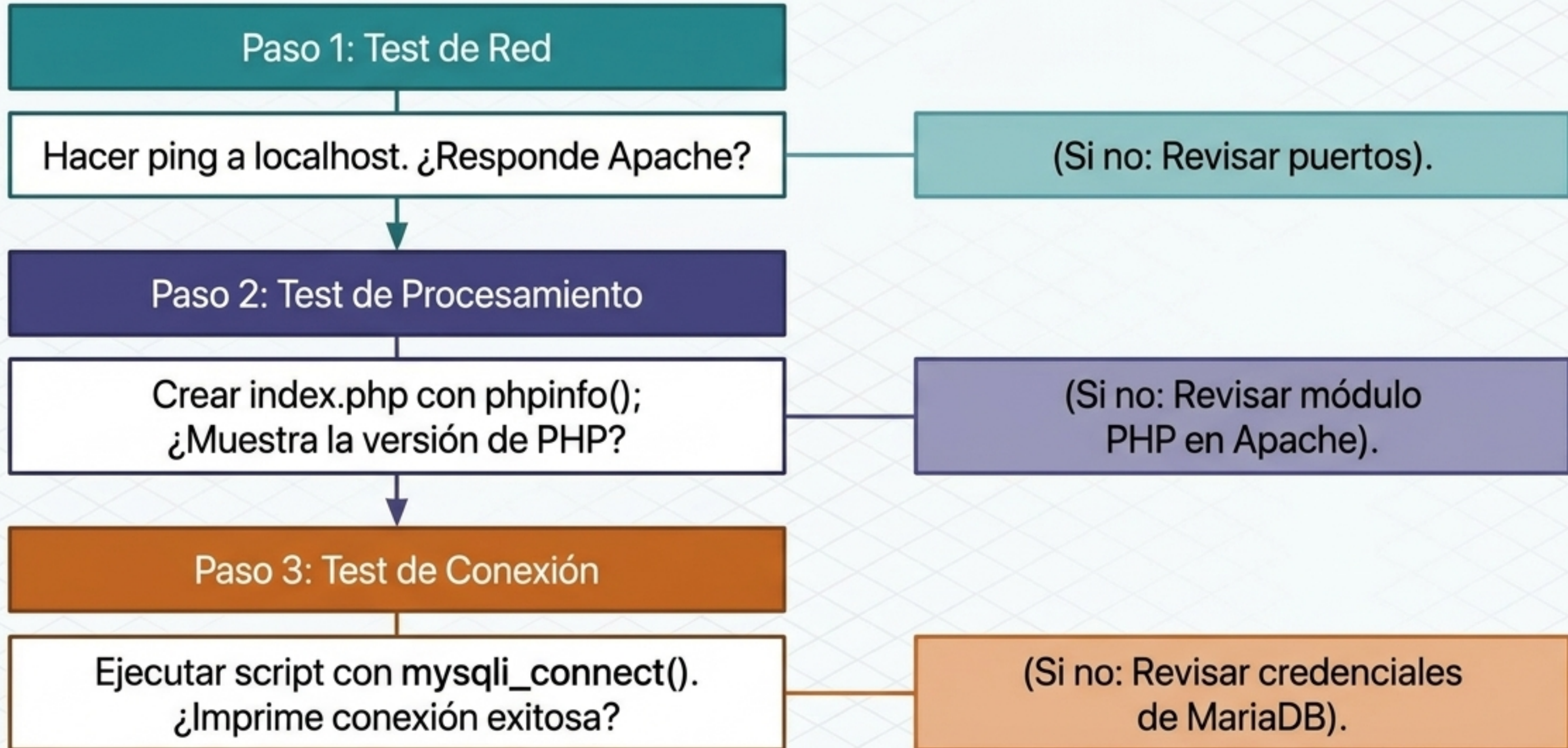
	Tradicional (Ej. XAMPP)	Moderno (Docker)
Arquitectura	Monolítica: todo en el mismo SO host	Microservicios: cada componente aislado
Reproducibilidad	Difícil de replicar exactamente en otro equipo	Perfectamente idéntico en desarrollo y producción
Conflictos	Propenso a choques de puertos con software local	Puertos aislados y mapeados virtualmente
Uso Ideal	Pruebas locales rápidas	Despliegues reales en Cloud y metodologías ágiles

Fase 3: Montaje Moderno con Docker



La instalación integrada moderna no es un instalador .exe. Es un orquestador que ejecuta entornos empaquetados e independientes que replican el servidor de producción al 100%.

Fase 4: Comprobación del Ecosistema (Testing)

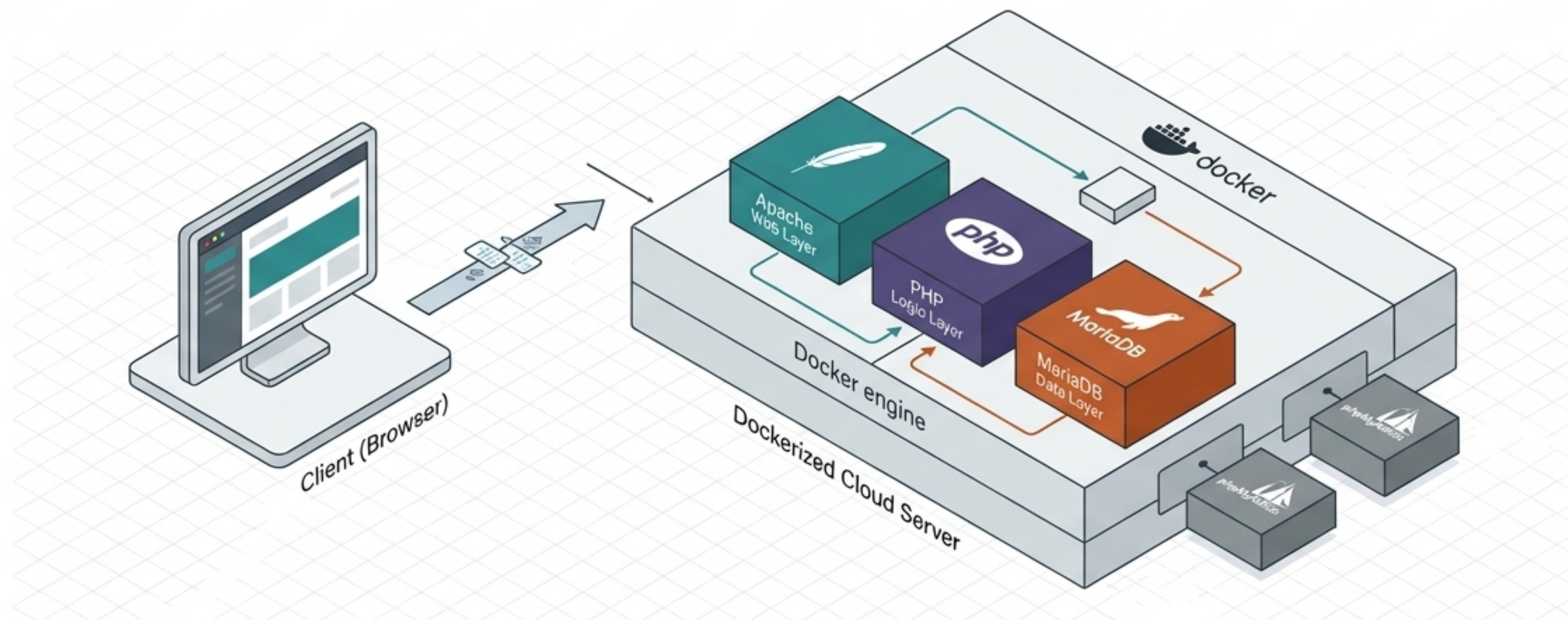


Fase 4: Documentación de la Infraestructura

En entornos de producción, la documentación es un elemento más del servidor.

- ✓ **Control de Versiones:** Registro exacto de versiones del SO, Apache, PHP y MariaDB (evita incompatibilidades futuras).
- ✓ **Esquemas de Datos:** Diagramas Entidad-Relación (ER) y restricciones de los campos de MariaDB.
- ✓ **Registro de Errores:** Procedimientos utilizados para reparar fallos durante el montaje.
- ✓ **Infraestructura como Código:** El archivo docker-compose.yml sirve ahora como la documentación viva de cómo se interconectan los contenedores.

Resumen del Ecosistema Cloud-Native



Planificar



El análisis de requisitos dicta la infraestructura.

Aislar



Desplegar módulos especializados evita arquitecturas monolíticas frágiles.

Contenerizar



Docker es la evolución natural del entorno de pruebas, garantizando escalabilidad.

Validar



Comprobar componentes y documentar la configuración asegura el mantenimiento.